Complete Guide: Fine-Tuning Llama 3.2 Locally with Ollama and LoRA

towardsdev.com/complete-guide-fine-tuning-llama-3-2-locally-with-ollama-and-lora-be7eb05314ff?gi=9c7a3b679b19

Saurabh Singh December 30, 2025

Press enter or click to view image in full size

Introduction

Imagine you've just hired a brilliant assistant who knows a lot about everything, but you need them to specialize in your specific domain — maybe customer support for your SaaS product, or generating content in your company's unique style. That's exactly what we're doing when we fine-tune Llama 3.2 with LoRA (Low-Rank Adaptation).

In this guide, we'll walk through training Llama 3.2 on your local machine using Ollama and LoRA, with detailed explanations and real examples.

What You'll Learn

- Setting up Ollama and required tools
- Understanding LoRA and why it's perfect for local training
- Preparing your training data
- Fine-tuning Llama 3.2 step-by-step
- Testing and using your custom model

Prerequisites

- A computer with at least 16GB RAM (32GB recommended)
- GPU with 8GB+ VRAM (optional but highly recommended)
- Basic command line knowledge
- Python 3.8 or higher

Part 1: Understanding the Concepts

What is Ollama?

Ollama is like Docker for AI models. It packages large language models into containers that you can run locally with simple commands. Instead of dealing with complex Python environments and model loading code, Ollama handles everything for you.

What is LoRA?

Think of a large language model as a massive textbook with billions of facts. Traditional fine-tuning is like rewriting the entire textbook — expensive and time-consuming. LoRA (Low-Rank Adaptation) is like adding sticky notes with corrections and additions throughout the book. You’re not changing the original text, just adding targeted modifications.

Key benefits of LoRA:

- Uses 90% less memory than full fine-tuning
- Trains 3–4x faster
- Works on consumer hardware
- Easy to swap between different adaptations

Part 2: Installation and Setup

Step 1: Install Ollama

For macOS/Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

For Windows: Download the installer from https://ollama.com/download

Verify installation:

```
ollama --version
```

Step 2: Pull Llama 3.2

Download the base model you want to fine-tune:

```
# For the 3B parameter model (lighter, faster)
ollama pull llama3.2:3b

# For the 1B parameter model (even lighter)
ollama pull llama3.2:1b
```

Test that it works:

```
ollama run llama3.2:3b "Hello, how are you?"
```

Step 3: Install Required Python Libraries

```
# Create a virtual environment
python -m venv llama-finetune
source llama-finetune/bin/activate  # On Windows: llama-finetune\Scripts\activate

# Install dependencies
pip install torch transformers datasets peft accelerate bitsandbytes
```

Part 3: Preparing Your Training Data

Understanding Data Format

Your training data should be in a conversational format. Here's what works best:

JSONL Format (JSON Lines):

```
{"prompt": "What is photosynthesis?", "completion": "Photosynthesis is the process by which plants use sunlight, water, and carbon dioxide to create oxygen and energy in the form of sugar."}
{"prompt": "How do plants make their own food?", "completion": "Plants make food through photosynthesis. They capture light energy with chlorophyll in their leaves and convert carbon dioxide and water into glucose and oxygen."}
{"prompt": "Why are leaves green?", "completion": "Leaves are green because they contain chlorophyll, a pigment that absorbs red and blue light but reflects green light, which is why we see them as green."}
```

Sample Dataset: Customer Support Bot

Let's create a real example. Say you're building a customer support bot for a fictional company "CloudStore" that sells cloud storage.

Create `training_data.jsonl`:

```
{"prompt": "How much storage do I get with the free plan?", "completion": "The CloudStore free plan includes 10GB of storage space. You can upgrade anytime to get more storage and additional features."}
{"prompt": "I forgot my password. What should I do?", "completion": "No problem! Click on 'Forgot Password' on the login page. Enter your email address, and we'll send you a password reset link within 5 minutes. Check your spam folder if you don't see it."}
{"prompt": "Can I share files with people who don't have CloudStore?", "completion": "Absolutely! You can create a shareable link for any file or folder. Go to the file, click the share icon, and copy the link. Anyone with the link can view or download the file, even without a CloudStore account."}
{"prompt": "What file types are supported?", "completion": "CloudStore supports all file types! You can upload documents, images, videos, audio files, archives, and more. There's no restriction on file formats."}
{"prompt": "Is my data encrypted?", "completion": "Yes, security is our top priority. All files are encrypted in transit using TLS and at rest using AES-256 encryption. Only you have access to your encryption keys."}
{"prompt": "How do I cancel my subscription?", "completion": "You can cancel anytime from your account settings. Go to Settings > Subscription > Cancel Plan. You'll retain access until the end of your billing period, and your data will remain available."}
{"prompt": "What's the difference between the Pro and Business plans?", "completion": "The Pro plan ($9.99/month) gives you 1TB storage and priority support. The Business plan ($19.99/month) includes 5TB storage, team collaboration features, admin controls, and advanced security options."}
{"prompt": "Can I recover deleted files?", "completion": "Yes! Deleted files go to the Trash where they're kept for 30 days. During this time, you can restore them anytime. After 30 days, files are permanently deleted."}
```

Save this as `training_data.jsonl` in your project folder.

Part 4: Preparing the Training Script

Understanding the Code Components

Before we write the full script, let’s understand what each part does:

1. : We load the base Llama 3.2 model
2. : We specify which parts of the model to adapt
3. : We format our data for training
4. : The actual fine-tuning process
5. : Converting to Ollama format

Create `finetune_llama.py`

```
import json
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from datasets import Dataset
from trl import SFTTrainer
```

```
# Configuration
MODEL_NAME = "meta-llama/Llama-3.2-3B"  # or use 1B version
DATASET_PATH = "training_data.jsonl"
OUTPUT_DIR = "./llama-cloudstore-support"# Step 1: Load the tokenizer
print("Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"# Step 2: Load the base model
print("Loading base model...")
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16,
    device_map="auto",
)# Step 3: Prepare model for training
model = prepare_model_for_kbit_training(model)# Step 4: Configure LoRA
# This is where the magic happens!
lora_config = LoraConfig(
    r=16,  # Rank of the update matrices (higher = more parameters, better quality)
    lora_alpha=32,  # Scaling factor (typically 2x the rank)
    target_modules=["q_proj", "v_proj"],  # Which attention layers to adapt
    lora_dropout=0.05,  # Dropout for regularization
    bias="none",  # Don't adapt bias terms
    task_type="CAUSAL_LM"
)# Apply LoRA to the model
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()  # Show how many parameters we're actually
training# Step 5: Load and prepare dataset
print("Loading dataset...")
def load_dataset_from_jsonl(file_path):
    data = []
    with open(file_path, 'r') as f:
        for line in f:
            item = json.loads(line)
            # Format as instruction-following
            text = f"### Human: {item['prompt']}\n### Assistant:
{item['completion']}"
            data.append({"text": text})
    return Dataset.from_list(data)dataset = load_dataset_from_jsonl(DATASET_PATH)#
Step 6: Set training parameters
training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=3,  # Number of complete passes through the data
    per_device_train_batch_size=4,  # Adjust based on your GPU memory
    gradient_accumulation_steps=4,  # Simulate larger batch size
    learning_rate=2e-4,  # Learning rate for LoRA
    fp16=True,  # Use mixed precision training
    logging_steps=10,
    save_strategy="epoch",
    optim="paged_adamw_8bit",  # Memory-efficient optimizer
)# Step 7: Initialize trainer
trainer = SFTTrainer(
    model=model,
```

```
    args=training_args,
    train_dataset=dataset,
    tokenizer=tokenizer,
    max_seq_length=512,
    dataset_text_field="text",
)# Step 8: Start training!
print("Starting training...")
trainer.train()# Step 9: Save the fine-tuned model
print("Saving model...")
model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)print(f"Training complete! Model saved to
{OUTPUT_DIR}")
```

Analogy: Understanding the LoRA Parameters

Think of the model as a large orchestra:

- : The number of musicians you're adding to modify the sound. Higher = more nuanced changes but needs more resources. Typically 8–64.
- : How loud these new musicians play relative to the original orchestra. Usually 2x the rank.
- : Which sections of the orchestra you're modifying (strings, brass, etc.). In transformers, we typically target attention layers.
- : Randomly "muting" some musicians during practice to prevent over-reliance on any single player (prevents overfitting).

Part 5: Running the Training

Execute Training

```
python finetune_llama.py
```

What you'll see:

```
Loading tokenizer...
Loading base model...
trainable params: 4,194,304 || all params: 3,217,644,544 || trainable%: 0.13
Loading dataset...
Starting training...
Epoch 1/3: 100%|██████████| 2/2 [00:45<00:00, 22.50s/it, loss=1.234]
Epoch 2/3: 100%|██████████| 2/2 [00:44<00:00, 22.12s/it, loss=0.876]
Epoch 3/3: 100%|██████████| 2/2 [00:43<00:00, 21.89s/it, loss=0.543]
Training complete!
```

Understanding the Output

- : Only ~0.13% of the model is being updated — that's LoRA's efficiency!

- : This number should decrease over time. Lower = better fit to your data
- : Depends on your hardware and dataset size

Part 6: Converting to Ollama Format

Create Modelfile

Create a file named `Modelfile`:

```
FROM llama3.2:3b

ADAPTER ./llama-cloudstore-supportPARAMETER temperature 0.7
PARAMETER top_p 0.9SYSTEM """
You are a helpful customer support agent for CloudStore, a cloud storage service.
Answer questions clearly and concisely. Be friendly and professional.
"""
```

Build Ollama Model

```
ollama create cloudstore-support -f Modelfile
```

Part 7: Testing Your Fine-Tuned Model

Interactive Testing

```
ollama run cloudstore-support
```

Try these prompts:

```
>>> How much storage do I get?
>>> I can't remember my password
>>> Can I share files with external users?
```

Programmatic Testing

Create `test_model.py`:

```
import subprocess
import json
```

```
def query_model(prompt):
    result = subprocess.run(
        ["ollama", "run", "cloudstore-support", prompt],
        capture_output=True,
        text=True
    )
    return result.stdout# Test queries
test_prompts = [
    "What's included in the free plan?",
    "How do I reset my password?",
    "Is my data secure?",
    "What's the difference between Pro and Business plans?"
]print("Testing fine-tuned model:\n")
for prompt in test_prompts:
    print(f"Q: {prompt}")
    response = query_model(prompt)
    print(f"A: {response}\n")
```

Part 8: Tips and Best Practices

Data Quality Matters

The golden rule: **Quality over Quantity**. 50 high-quality examples beat 500 mediocre ones.

Join Medium for free to get updates from this writer.

Remember me for faster sign in

Good example:

```
{"prompt": "How do I upgrade?", "completion": "To upgrade your CloudStore plan, go to
Settings > Subscription > Upgrade. Choose your preferred plan and enter your payment
details. The upgrade takes effect immediately, and you'll only pay the prorated
difference for the current billing period."}
```

Bad example:

```
{"prompt": "upgrade", "completion": "go to settings"}
```

Hyperparameter Tuning

Start with these settings and adjust:

- Reduce `per_device_train_batch_size` to 1 or 2
- Increase `gradient_accumulation_steps`
- Increase `learning_rate` to 3e-4 or decrease `epochs`
- Increase `lora_dropout` or reduce `epochs`

Signs of Good Training

- Loss decreases steadily
- Model responses are coherent
- Answers match your domain knowledge
- No repetitive or nonsensical outputs

Common Issues and Solutions

Issue: Out of Memory Error

```
# Solution: Reduce batch size
per_device_train_batch_size=1
gradient_accumulation_steps=8
```

Issue: Model gives generic responses

```
# Solution: More training data or increase LoRA rank
r=32
lora_alpha=64
```

Issue: Model responses are too similar to training data

```
# Solution: Increase dropout or reduce epochs
lora_dropout=0.1
num_train_epochs=2
```

Part 9: Advanced Topics

Multi-Turn Conversations

Format your data with conversation history:

```
{"prompt": "How much does CloudStore cost?\n### Assistant: We offer three plans: Free (10GB), Pro ($9.99/month, 1TB), and Business ($19.99/month, 5TB).\n### Human: What if I need more than 5TB?", "completion": "For storage needs exceeding 5TB, we offer custom Enterprise plans. Contact our sales team at enterprise@cloudstore.com, and we'll create a tailored solution with dedicated support and custom pricing."}
```

Evaluation Metrics

Track these metrics to measure improvement:

```
from sklearn.metrics import accuracy_score
import numpy as np
```

```
def evaluate_model(test_data):
    correct = 0
    total = len(test_data)         for item in test_data:
        response = query_model(item['prompt'])
        # Simple keyword matching (use more sophisticated methods in production)
        if any(keyword in response.lower() for keyword in item['expected_keywords']):
            correct += 1         accuracy = correct / total
    print(f"Accuracy: {accuracy:.2%}")
    return accuracy
```

Version Control

Keep track of your experiments:

```
# Tag your models
ollama create cloudstore-support:v1.0 -f Modelfile
ollama create cloudstore-support:v1.1 -f Modelfile-improved

# Compare versions
ollama run cloudstore-support:v1.0 "test prompt"
ollama run cloudstore-support:v1.1 "test prompt"
```

Part 10: Deployment Considerations

API Server

Serve your model as an API:

```
from flask import Flask, request, jsonify
import subprocess

app = Flask(__name__)@app.route('/chat', methods=['POST'])
def chat():
    prompt = request.json['prompt']
    response = subprocess.run(
        ["ollama", "run", "cloudstore-support", prompt],
        capture_output=True,
        text=True
    )
    return jsonify({"response": response.stdout})if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Performance Optimization

- Use quantization for faster inference
- Batch requests when possible
- Cache common queries
- Consider GPU acceleration for production

Conclusion

You've now learned how to fine-tune Llama 3.2 locally using LoRA and Ollama! This approach gives you:

- : Your data never leaves your machine
- : Tailored responses for your specific use case
- : No API fees or cloud costs
- : Perfect for sensitive domains

Next Steps

1. Expand your training dataset with more examples
2. Experiment with different LoRA configurations
3. Try fine-tuning for multiple domains
4. Build a evaluation framework for continuous improvement

Resources

- Ollama Documentation: https://ollama.com/docs
- LoRA Paper: "LoRA: Low-Rank Adaptation of Large Language Models"
- Hugging Face PEFT: https://huggingface.co/docs/peft

Happy fine-tuning! 🚀